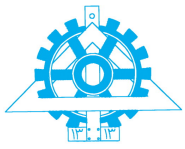


پروژه سوم

اندازه گیری اینترنت



ابزار لینوکس

ـ سیدعلی آقامیری

استاد دکتر احمد خونساری

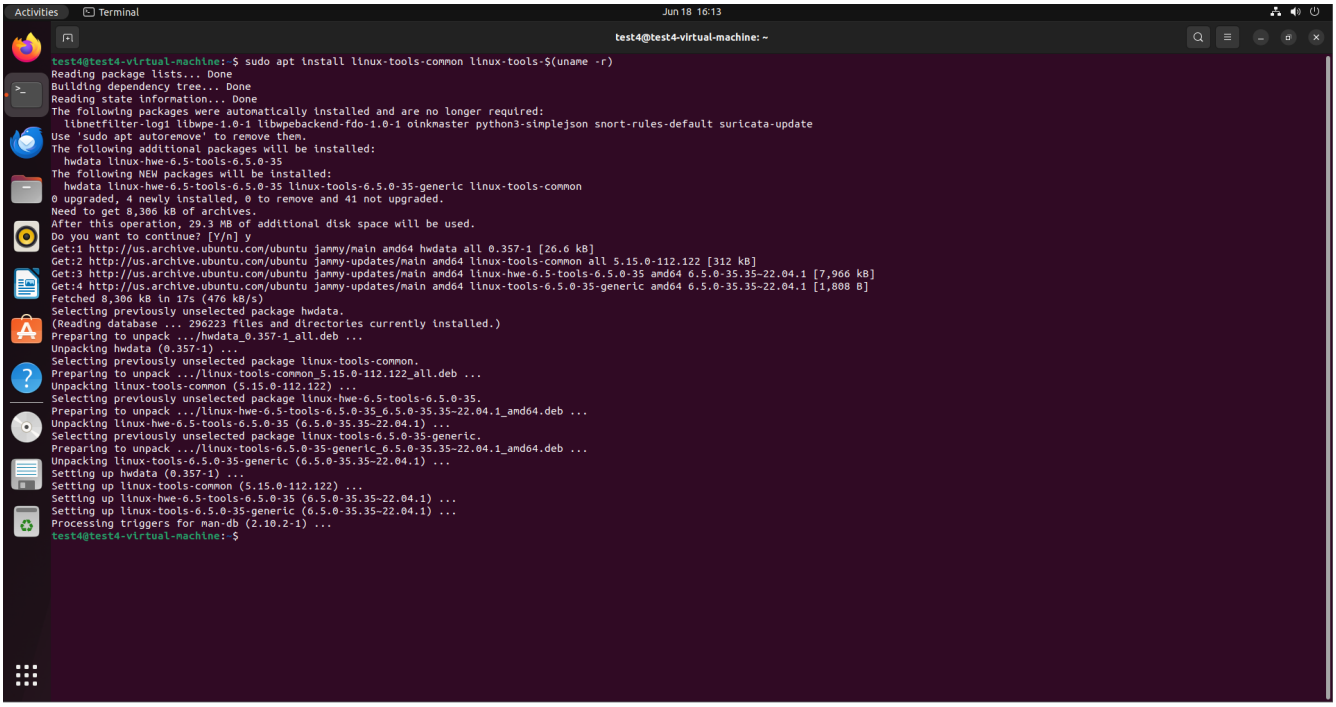
پروژه سوم

مراحل نصب ابزارها

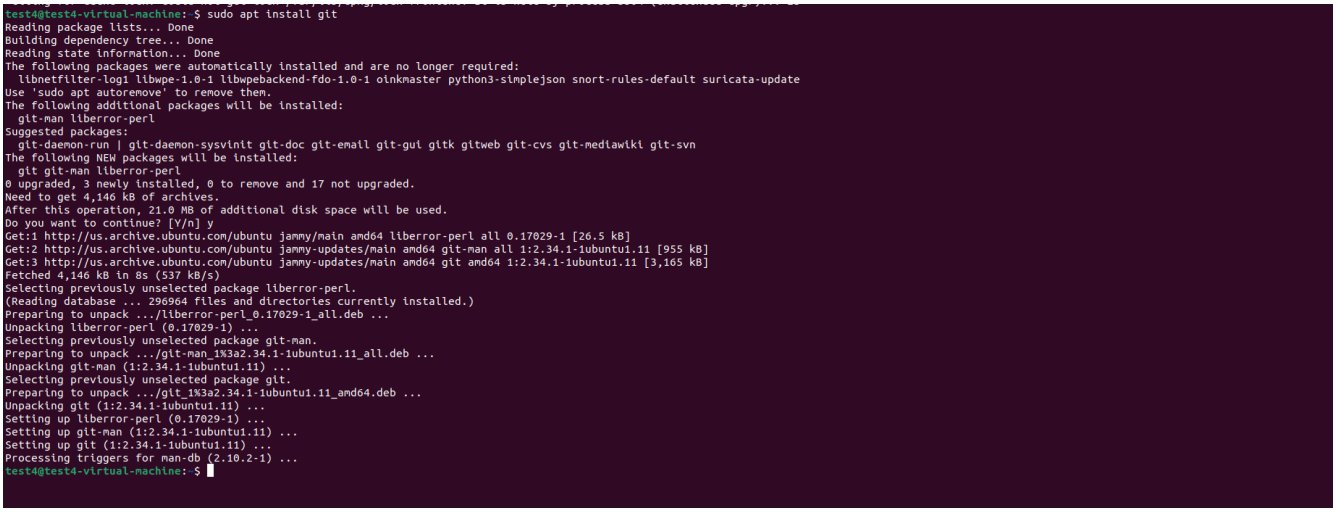
برای نصب ابزارهای Perf و Flame Graph ابتدا باید از به‌روزبودن سیستم اطمینان حاصل کنید. سپس با استفاده از دستورات زیر می‌توانید این ابزارها را نصب کنید:

ابتدا برای نصب Perf:

```
sudo apt update
sudo apt install linux-tools-common linux-tools-$(uname -r)
```




```
sudo apt install git
```



سپس برای کلون کردن مخزن Flame Graph از دستور زیر استفاده کنید:

```
git clone https://github.com/brendangregg/FlameGraph.git
```



با انجام مراحل فوق، ابزارهای Perf و Flame Graph بر روی سیستم نصب شدند. این ابزارها به ما امکان مانیتورینگ دقیق و تحلیل عملکرد پردازنده و برنامه‌ها را می‌دهند. با استفاده از Perf می‌توان داده‌های مختلفی از عملکرد CPU جمع‌آوری کرد و سپس با استفاده از Flame Graph این داده‌ها را به صورت بصری و قابل فهمی نمایش داد. این ابزارها به بهبود عملکرد برنامه‌ها و شناسایی مشکلات آنها کمک زیادی می‌کنند.

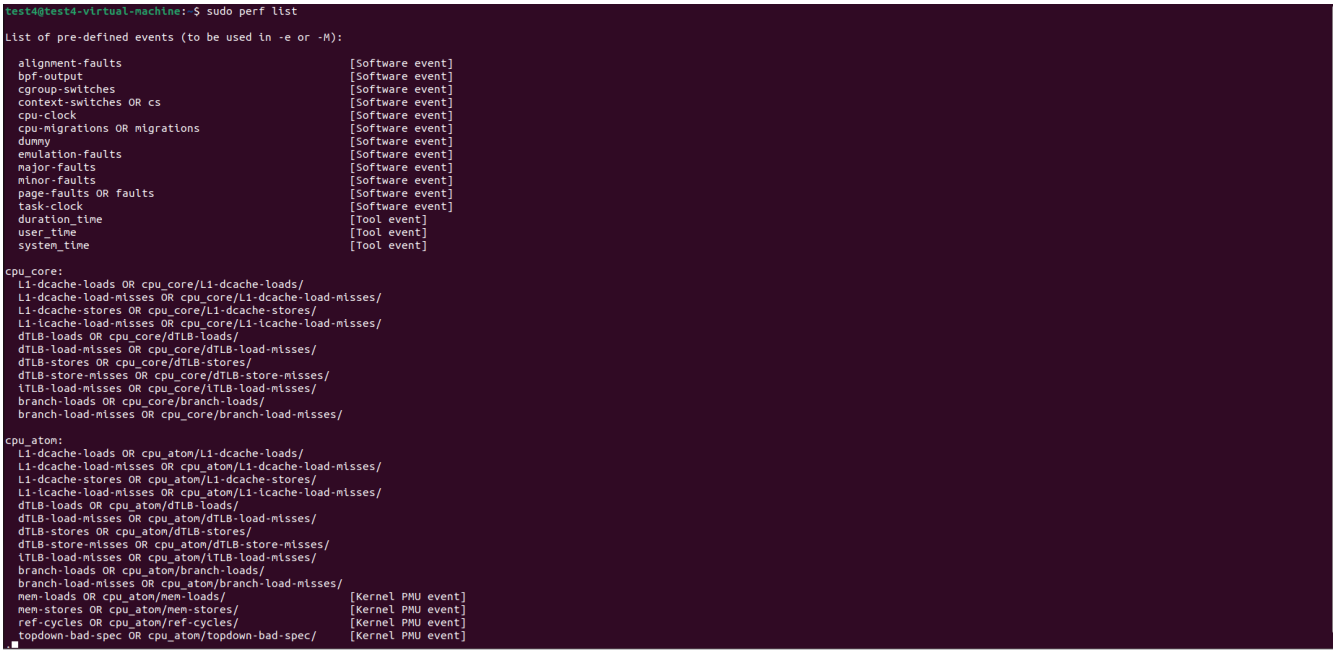
دستورات پرکاربرد Perf

در این بخش چند مورد از دستورات پرکاربرد Perf را بررسی و نتایج اجرای آنها را گزارش می‌کنیم.

list perf

دستور perf list یکی از دستورات پرکاربرد ابزار Perf است که تمامی رویدادهای قابل مانیتورینگ را نمایش می‌دهد. این رویدادها شامل رویدادهای نرم‌افزاری، رویدادهای مربوط به ابزار و رویدادهای مربوط به واحدهای پردازنده (CPU) هستند. به عنوان مثال، برخی از رویدادهای نرم‌افزاری شامل alignment-faults، context-switches و cpu-clock می‌باشند. همچنین، رویدادهای مربوط به ابزار شامل duration time، user time و system time هستند.

```
sudo perf list
```

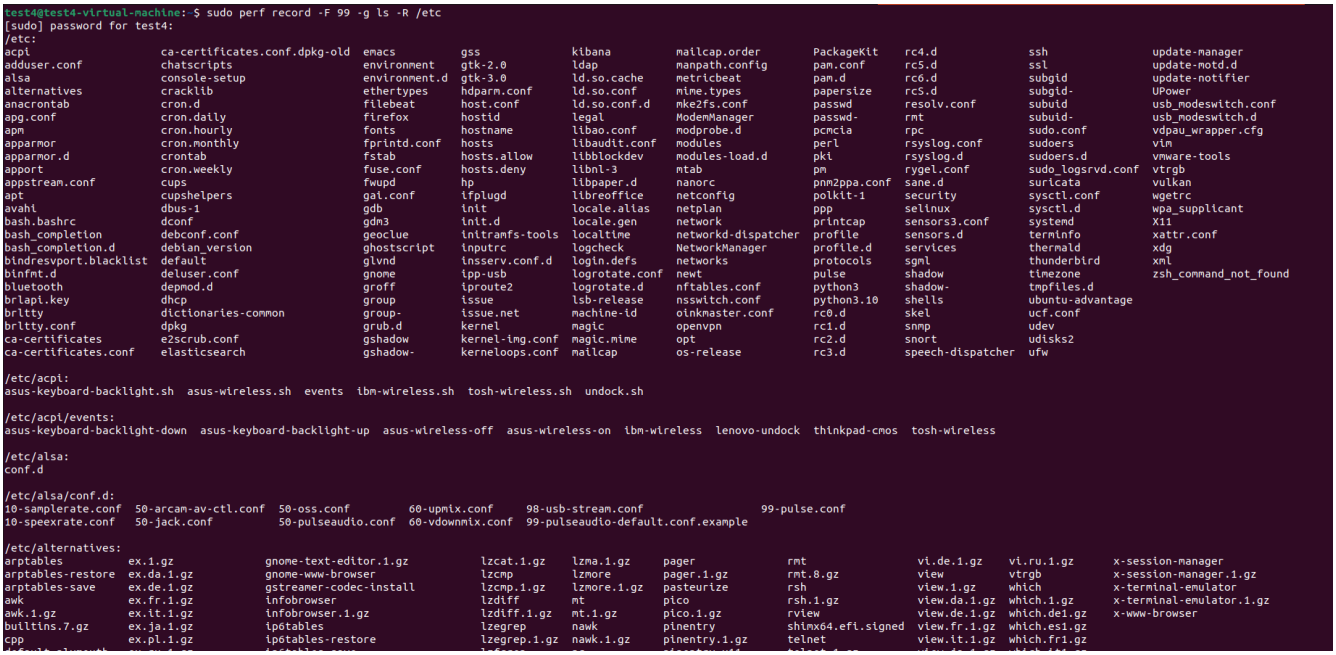


رویدادهای مربوط به واحدهای پردازنده نیز به دو دسته cpu core و cpu atom تقسیم می‌شوند. هر یک از این دسته‌ها شامل مجموعه‌ای از رویدادهای مختلف هستند که برای مانیتورینگ عملکرد واحدهای مختلف پردازنده استفاده می‌شوند. به عنوان مثال، در دسته cpu core می‌توان به رویدادهای L1-dcache-loads، dTLB-load-misses و branch-load-misses اشاره کرد. در دسته cpu atom نیز رویدادهای مشابهی مانند L1-dcache-loads، dTLB-load-misses و branch-load-misses وجود دارند. این رویدادها به برنامه‌نویسان و مدیران سیستم کمک می‌کنند تا عملکرد سیستم را به صورت دقیق‌تری تحلیل و بهبود بخشند.

record perf

دستور perf record یکی از دستورات اصلی ابزار Perf است که برای جمع‌آوری داده‌های پروفایلینگ از برنامه‌ها استفاده می‌شود. با استفاده از این دستور می‌توان داده‌های عملکردی برنامه‌ها را ضبط کرد و سپس با دستورات دیگر مانند perf report تحلیل نمود. در این مثال، از دستور perf record برای ضبط عملکرد دستور ls -R /etc استفاده شده است. این دستور تمامی فایل‌ها و دایرکتوری‌های موجود در مسیر /etc را به صورت بازگشتی لیست می‌کند و داده‌های مربوط به عملکرد این عملیات توسط perf ضبط می‌شود.

```
sudo perf record -F 99 -g ls -R /etc
```



نتیجه اجرای دستور perf record نشان‌دهنده لیست کاملی از فایل‌ها و دایرکتوری‌های موجود در مسیر /etc است. این لیست شامل فایل‌های سیستمی مختلفی مانند adduser.conf، bash.bashrc، ca-certificates.conf و بسیاری دیگر می‌باشد. استفاده از این دستور به ما کمک می‌کند تا عملکرد سیستم را در حین اجرای این نوع دستورات معمولی تحلیل کرده و مشکلات احتمالی را شناسایی کنیم. این داده‌ها می‌توانند برای بهینه‌سازی عملکرد سیستم و برنامه‌ها بسیار مفید باشند.

perf-report

با استفاده از این دستور زیر اطلاعات جامعی ارائه می‌شود که برای تحلیل عملکرد برنامه‌ها بسیار مفید است:

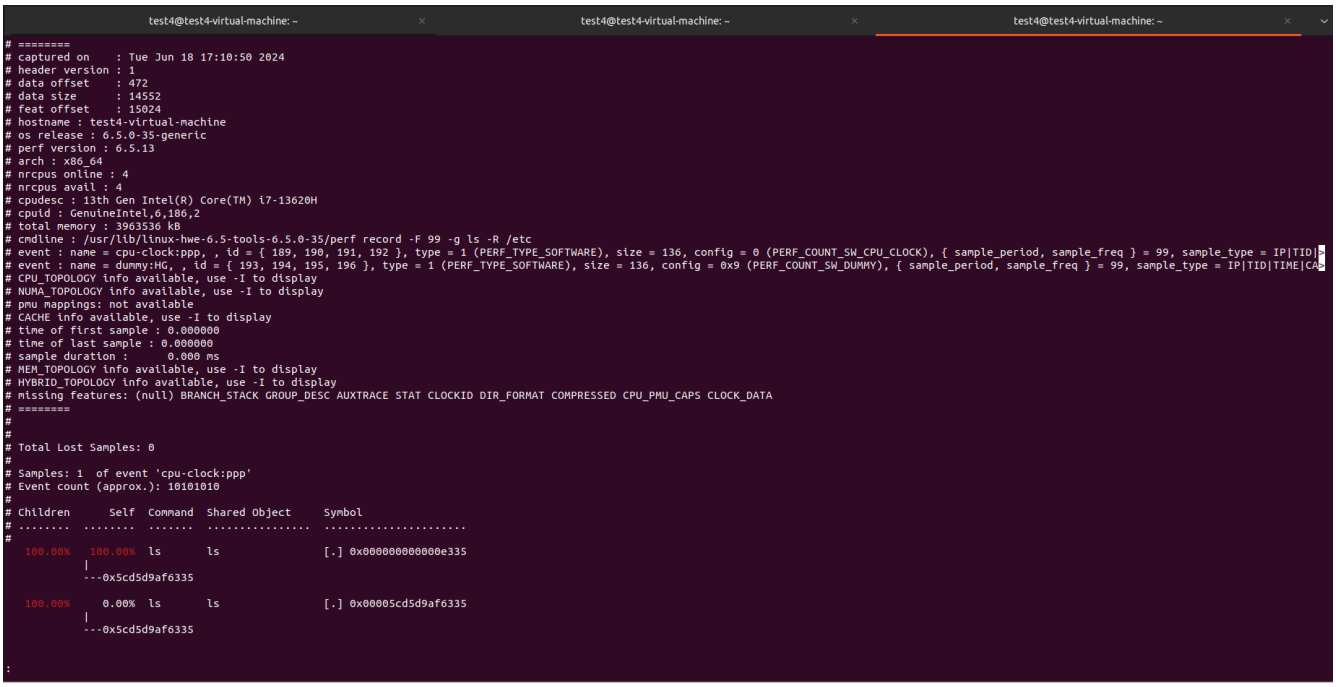
اطلاعات زمانی:

در ابتدا، تاریخ و زمانی که داده‌ها ضبط شده‌اند، نمایش داده می‌شود. این اطلاعات اهمیت بسیاری دارند تا بتوان داده‌های ضبط شده را به زمان وقوع رویدادها در برنامه مرتبط کرد.

اطلاعات سیستمی:

بخش بعدی شامل جزئیات سیستمی می‌باشد که شامل نام میزبان، نسخه سیستم عامل، و نسخه ابزار perf است. این اطلاعات مفید هستند تا بررسی‌ها و تفسیرهای مربوط به عملکرد برنامه را بهبود بخشیده و از جزئیات فنی نظیر نوع پردازنده و تعداد هسته‌های فعال بهره برده شود.

```
sudo perf report --header
```



این دستور اساسی است برای تحلیل و بررسی داده‌های ضبط شده توسط perf، که به مهندسان و توسعه‌دهندگان کمک می‌کند تا عملکرد برنامه‌ها را بهینه‌سازی و بهبود دهند.

perf-annotate

دستور زیر در واقع برای تحلیل عملکرد یک برنامه با استفاده از perf استفاده می‌شود. این دستور اطلاعاتی را به کاربر نشان می‌دهد که در فرآیند آنالیز با perf بوجود آمده است. البته بسته به سیستم و شرایط خاص، ممکن است اطلاعات بیشتری نیز نمایش داده شود که به نکات زیر مربوط می‌شود:

```
sudo perf annotate
```

```
test4@test4-virtual-machine:~$ sudo perf annotate --stdio -v
build id event received for /usr/bin/ls: 36b86f957a1be53733633d184c3a3354f3fc7b12 [20]
build id event received for [vdso]: c3b6337f80a46b720a90fb56864441cf0aa37fce [20]
socket_id number is too big.You may need to upgrade the perf tool.
```

در کل، این دستور به کاربر اطلاعات تکمیلی ارائه می‌دهد که برای فهم بهتر و استفاده بهینه از perf مفید است، از جمله اطلاعاتی درباره شناسه ساخت (build id) و نیاز به ارتقاء perf برای پشتیبانی از شناسه‌های socket id بزرگ‌تر

نتیجه‌گیری

ابزار Perf یک ابزار بسیار مفید برای تحلیل و بهبود عملکرد برنامه‌ها در سیستم‌های لینوکس است. با استفاده از دستورات مختلف این ابزار می‌توانیم داده‌های پروفایلینگ دقیقی جمع‌آوری کرده و عملکرد برنامه‌ها را بهبود بخشیم.

قسمت سوم

برای اجرای یک برنامه در perf باید مراحل زیر را انجام دهیم:

۱. ابتدا با استفاده از دستور زیر برنامه مورد نظر را شروع به اجرا و رکورد می‌کنیم با استفاده از perf:

```
sudo perf record -F 99 -g -o main.data - ./program1
```

در این دستور F- 99 نشان دهنده زمان نمونه‌برداری است و g- جهت اجرای توابع داخل برنامه است. بعد از اجرای دستور فوق فایل main.data تشکیل می‌شود.

```
ubuntu@ubuntu-1-2:~/Downloads/Project3$ sudo perf record -o main.data -- ./program1
[ perf record: Woken up 59 times to write data ]
[ perf record: Captured and wrote 14.897 MB main.data (390232 samples) ]
ubuntu@ubuntu-1-2:~/Downloads/Project3$
```

رکورد کردن داده‌ها با perf: این مرحله با استفاده از دستور record perf شروع می‌شود که با اجرای آن برنامه مورد نظر را به همراه داده‌های عملکردی آن رکورد می‌کنیم. پارامتر F- 99 مشخص می‌کند که نمونه‌برداری با چه فرکانسی انجام شود و g- برای جمع‌آوری اطلاعات کامل تابعی است که به همراه نام برنامه استفاده می‌شود. فایل main.data که با اجرای این دستور تشکیل می‌شود، حاوی داده‌های اجرای برنامه است.

۲. در این مرحله، فایل data. باید به فایل مناسب برای استفاده در Flamegraph تبدیل شود که به صورت زیر به فایل perf. تبدیل می‌شود:

```
sudo perf script -i main.data > main.perf
```

تبدیل فرمت فایل با perf script: در مرحله دوم، با استفاده از دستور script perf، فایل main.data که در مرحله قبل ایجاد شده است، به یک فرمت مناسب برای استفاده در Flamegraph تبدیل می‌شود و به فایل main.perf ذخیره می‌شود. این فایل شامل اطلاعات دقیق‌تری از عملکرد توابع و توانایی‌های برنامه است که برای تجزیه و تحلیل بیشتر استفاده می‌شود.

```
ubuntu@ubuntu-1-2:~/FlameGraph$ ./stackcollapse-perf.pl perfs/main.perf > main.floded
Unrecognized line:          5876161563e8 mai at ./stackcollapse-perf.pl line 429, <>
line 1484290.
ubuntu@ubuntu-1-2:~/FlameGraph$ ./flamegraph.pl main.floded > main.svg
```

۳. در مرحله سوم، ما فایل مورد نظر را به پوشه Flamegraph منتقل کرده و سپس آن را به یک فایل با فرمت `folded` تبدیل می‌کنیم:

```
./stackcollapse-perf.pl main.perf > main.folded
```

تبدیل به فرمت folded با استفاده از stackcollapse-perf.pl: در این مرحله، با استفاده از ابزار `stackcollapse-perf.pl`، فایل `main.perf` به یک فرمت `folded` تبدیل می‌شود. این فرمت شامل یک لیست از توابع و زمان صرف شده توسط هر کدام است، که برای استفاده در تولید Flamegraph لازم است.

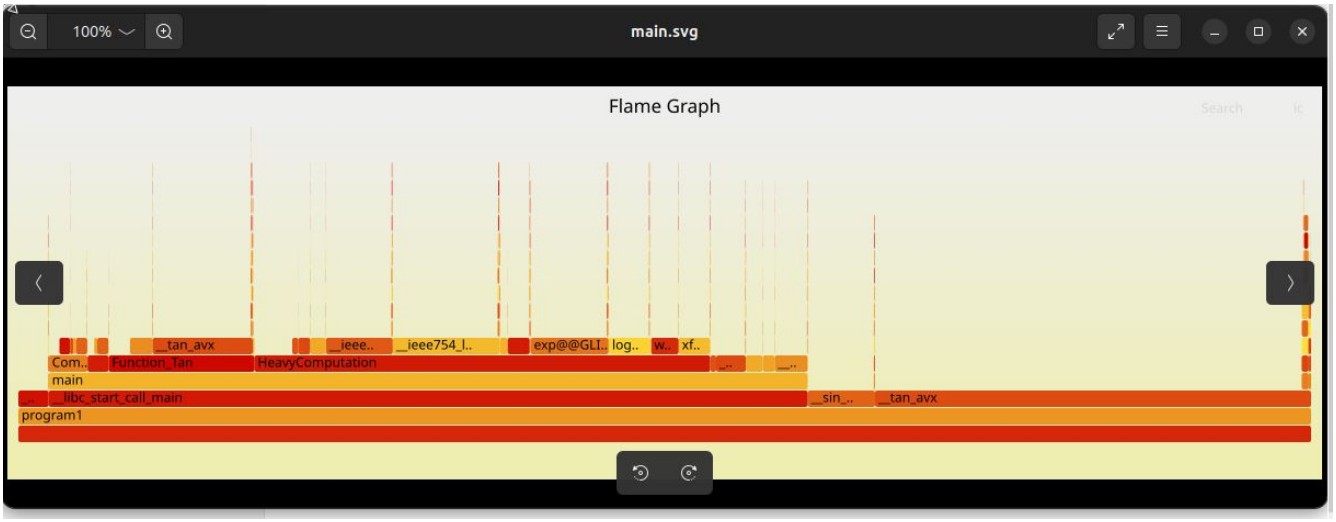
۴. حال می‌توانیم فایل تصویری نهایی را تولید کنیم با استفاده از دستور زیر:

```
./flamegraph.pl main.folded > flamegraph.svg
```



تولید نمودار Flamegraph: در این مرحله با استفاده از ابزار `flamegraph.pl`، فایل `main.folded` به یک فایل `flamegraph.svg` تبدیل می‌شود که شامل نمودار تابعی از عملکرد برنامه است. این نمودار به طور گسترده‌ای برای تحلیل و بصری‌سازی عملکرد برنامه و شناسایی نقاط ضعف و بهبود پذیری در کدهای برنامه‌نویسی استفاده می‌شود.

۵. نتیجه نهایی تصویر تولید شده به شکل زیر است:



تحلیل نمودار فوق و مشخص کردن گلوگاه نرم افزار جهت بهینه سازی تابع و کد : تصویر نهایی از چندین مستطیل (اسپن) تشکیل شده که از پایین به بالا چیده شده اند در ابتدا برنامه اجرا شده که بقیه توابع و موارد در درون آن هستند (در بالای آن بقیه موارد فراخوانی شده اند). کتابخانه libc که برای فراخوانی تابع main برنامه استفاده شده در بالای آن قرار دارد و بعد از آن تابع اصلی برنامه قرار دارد.

همانطور که میدانیم در نمودار فلیم گراف محور افقی محور زمان است و هرچه عرض اسپن های مشخص شده بیشتر باشد یعنی زمان بیشتری در آنجا برنامه در حال اجرا است (در نمودار فوق که مربوط به عملکرد cpu است) به همین دلیل گلوگاه های سیستم در بیشتر مواقع جایی است که زمان اجرای تابع زیاد باشد به همین دلیل بهینه سازی آن قسمت از توابع بسیار معقول تر و منطقی تر است.

بعد از تابعmain function-tan و heavyComputation و... قرار دارد که این دو مورد از نظر زمان اجرا گلوگاه های اصلی توابع فراخوانی شده از main هستند و بهینه سازی باید در این قسمت ها اجرا شود. بر روی آنها نیز توابع و متدهایی که فراخوانی کرده اند مشخص است. در تابع funciont-tan بیشترین زمان مربوط به tan.avx است که باید برای بهینه تر شدن بررسی گردد. و به همین ترتیب میتوان عملکرد کلی سیستم را بررسی نمود.

تذکر : برای اینکه بتوانیم خروجی memory را نیز داشته باشیم از دستورات perf که عبارت است از minor-faults و faults است استفاده میکنیم در این دستورات تعداد پیج فالت های مینور و ماژور مشخص میشود در برنامه مورد نظر (در اینجا ما ازدستور faults استفاده کرده ایم)

۶. برای تولید این نمودار ها هم موارد فوق و هم دستورات زیر را تکرار میکنیم:

۱. **رکورد کردن داده‌ها با perf:** برای شروع، با استفاده از دستور زیر، برنامه مورد نظر را با استفاده از ابزار perf اجرا و داده‌های عملکرد آن را ثبت می‌کنیم:

```
sudo perf record -e faults -F 99 -g -o faults.data ./program1
```

در این دستور، پارامتر `-e faults` برای رکورد خطاها، `-F 99` برای تعیین فرکانس نمونه‌برداری، `-g` برای جمع‌آوری اطلاعات کامل تابعی و `-o faults.data` برای ذخیره داده‌ها به عنوان فایل `faults.data` استفاده شده است.

۲. **تبدیل فرمت فایل با perf script:** پس از رکورد کردن داده‌ها، فایل `faults.data` باید به فرمت `perf` تبدیل شود تا بتوان اطلاعات دقیق‌تری از عملکرد برنامه را بررسی کرد. دستور زیر این کار را انجام می‌دهد:

```
sudo perf script -i faults.data > faults.perf
```

در این مرحله، اطلاعات جمع‌آوری شده در فایل `faults.data` به فرمت `perf` تبدیل شده و در فایل `faults.perf` ذخیره می‌شود.

۳. **تبدیل به فرمت folded با استفاده از stackcollapse-perf.pl:** برای تحلیل و تجزیه داده‌های موجود در فایل `faults.perf`، از ابزار stackcollapse-perf.pl استفاده می‌کنیم تا این فایل را به یک فرمت `folded` تبدیل کنیم و بتوانیم لیستی از توابع و زمان صرف شده توسط هر کدام را مشاهده کنیم:

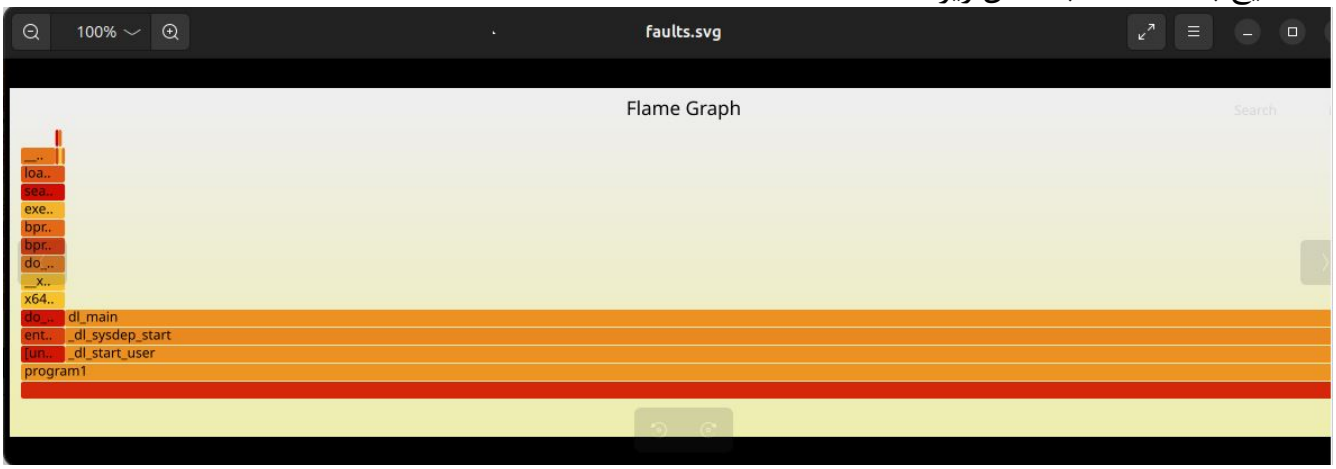
```
./stackcollapse-perf.pl faults.perf > faults.folded
```

۴. **تولید نمودار Flamegraph:** در مرحله آخر، با استفاده از ابزار flamegraph.pl، فایل `faults.folded` به یک فایل نمودار با پسوند `svg` تبدیل می‌شود که نموداری از عملکرد برنامه بر اساس تابع‌ها را نمایش می‌دهد:

```
./flamegraph.pl faults.folded > faults.svg
```

این نمودار Flamegraph برای بررسی عمیق‌تر عملکرد برنامه، تحلیل خطاها و بهبود کارایی برنامه مفید است.

۷. نتایج بدست آمده به شکل زیر است :



از نظر پیج فالت و miss شدن صفحات برنامه در مموری همانطور که در این نمودار مشخص است صفحات اصلی که باعث پیج فالت شده اند و زمان زیادی را به خود اختصاص داده اند تابع main برنامه است و بقیه توابع و فانکشن ها تقریبا به اندازه هم هستند و نقطه بحرانی خاصی مشاهده نمیشود.

قسمت چهارم

مرحله ۱:

در ابتدا، برنامه‌ای برای مرتب‌سازی ایجاد می‌کنیم که شامل سه تابع است. این توابع در یک فایل cpp. ذخیره می‌شوند. سپس با استفاده از دستور زیر، فایل برنامه را کامپایل می‌کنیم:

```
g++ -g -o sortnew sortnew.cpp
```

در این دستور، پارامتر g- نشان دهنده افزودن اطلاعات debug است و sortnew -o فایل اجرایی را به نام sortnew ایجاد می‌کند که از فایل sortnew.cpp ساخته شده است.

مرحله ۲:

با استفاده از برنامه perf، داده‌های عملکرد برنامه را به یک فایل اسکریپت با پسوند perf. تبدیل می‌کنیم. سپس به مراحل قبلی، از جمله تبدیل فایل perf. به فایل folded. و سپس به تولید فایل نمودار SVG ادامه می‌دهیم تا تحلیل و نمایش بهتری از عملکرد برنامه‌مان داشته باشیم.

این مراحل به ما کمک می‌کنند تا با استفاده از ابزارهای perf و تحلیل‌های آن، عملکرد و بهینه‌سازی برنامه‌هایمان را بهبود بخشیم و مشکلات عملکردی را شناسایی کنیم.

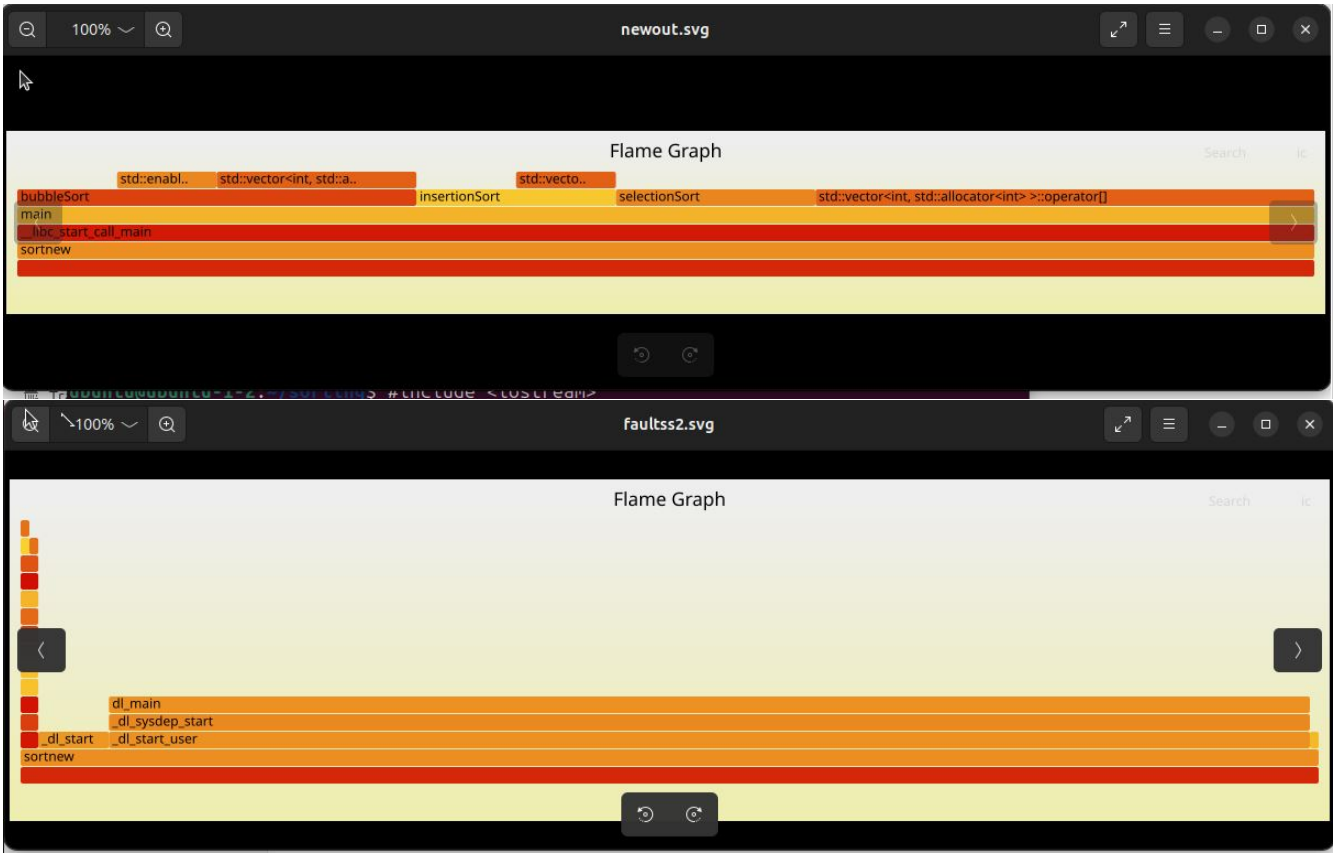
```
ubuntu@ubuntu-1-2:~/FlameGraph$ ./stackcollapse-perf.pl newout.perf > newout.floded
ubuntu@ubuntu-1-2:~/FlameGraph$ ./flamegraph.pl newout.floded > newout.svg
ubuntu@ubuntu-1-2:~/FlameGraph$
```

```
ubuntu@ubuntu-1-2:~/sorting$ sudo perf record -g ./sort2
Insertion Sort: 11 12 22 25 34 64 90
Duration: 1 microseconds
Selection Sort: 11 12 22 25 34 64 90
Duration: 1 microseconds
Bubble Sort: 11 12 22 25 34 64 90
Duration: 2 microseconds
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 0.010 MB perf.data (7 samples) ]
```

```
ubuntu@ubuntu-1-2:~/sorting$ sudo perf record -e faults -F 99 -g -o faultss.data -- ./
sortnew
Insertion Sort: 0.021114 seconds
Selection Sort: 0.027175 seconds
Bubble Sort: 0.05969 seconds
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 0.010 MB faultss.data (5 samples) ]
ubuntu@ubuntu-1-2:~/sorting$
```

پی نوشت : کد نوشته شده الصاق میگردد

مرحله ۳. نمودار خروجی سی پیو و پیج ممووری به شکل زیر است:



همانطور که در تصویر مشخص است، تمامی توابع روی تابع والد خود (main) قرار دارند. در تصویر میبینیم که sort insertion زمان کمتری را از سی پیو میگیرد و بعد از آن سلکشن سورت که تقریبا زمان اجرای هم اندازه با inserctionsort دارد. بیشترین زمان اجرا نیز مربوط به الگوریتم bubblesort میباشد که همانطور که میدانیم بهینگی کمتری نسبت به الگوریتم های مرتب سازی دارد. همچنین مقدار دهی آرای و فراخوانی مقادیر زمان زیادی را اشغال کرده است که قابل بهینه شدن میباشد.

بنابر این طبق نمودار فوق دربین الگوریتم های مشخص شده تایم بیشتر را روی بهینه کردن الگوریتم bub-blesort میگذاریم و باتل نک سیستم این قسمت میباشد.

قسمت امتیازی

مراحل نصب و استفاده از VTune: Intel

دانلود و نصب VTune: Intel

نسخه مناسب برای ویندوز را از وب‌سایت VTune Intel دانلود می کنیم.

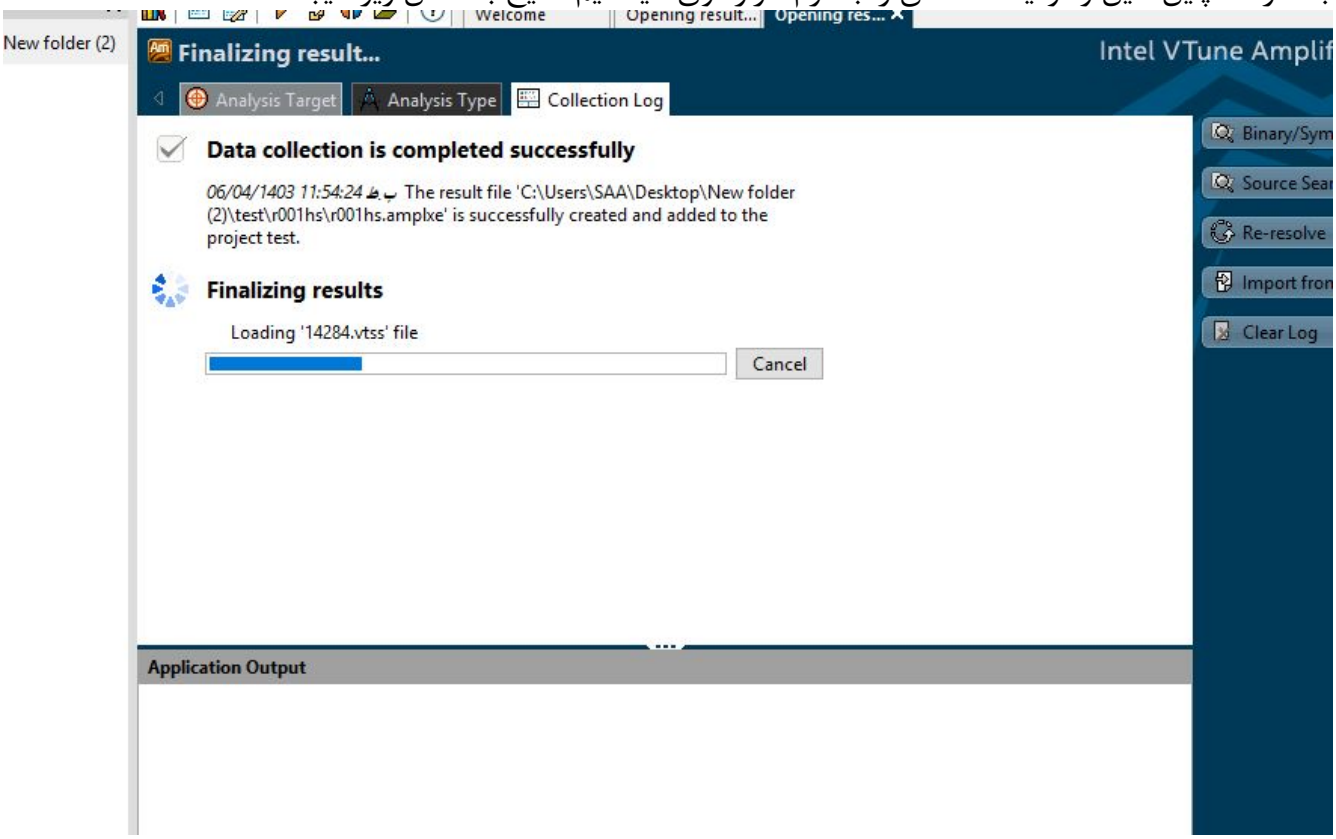
مراحل نصب را با اجرای فایل نصب کننده و پیروی از دستورالعمل‌های موجود انجام می دهیم. تحلیل عملکرد برنامه:

برنامه‌ای که می‌خواهید تحلیل کنید را انتخاب می کنیم. VTune Intel را باز کنید. یک پروژه جدید ایجاد می کنیم و برنامه مورد نظر را به پروژه اضافه می کنیم. نوع تحلیل مورد نظر (مثلاً تحلیل CPU، حافظه، I/O و ...) را انتخاب می کنیم. تحلیل را شروع می کنیم و منتظر می مانید تا عملیات پروفایلینگ به پایان برسد.

بصری‌سازی نتایج:

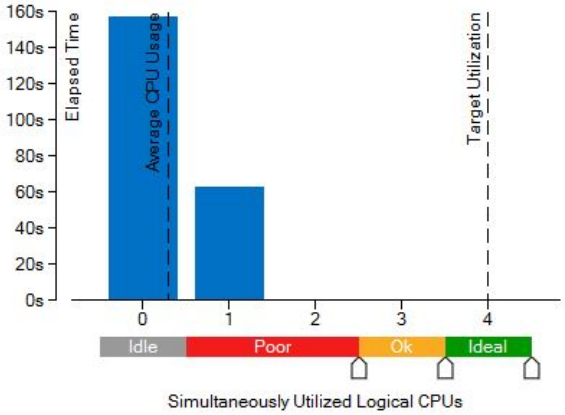
نتایج تحلیل را بررسی می کنید. VTune Intel گراف‌ها و نمودارهای مختلفی را برای نشان دادن عملکرد برنامه و مشکلات احتمالی ارائه می‌دهیم. می‌توانید از گراف‌های تولید شده توسط VTune برای شناسایی و بهبود بخش‌هایی از برنامه که نیاز به بهینه‌سازی دارند استفاده می کنیم.

بعد از کامپایل فایل و تولید exe. آن را به نرم افزار فوق میدهیم نتایج به شکل زیر میباشد

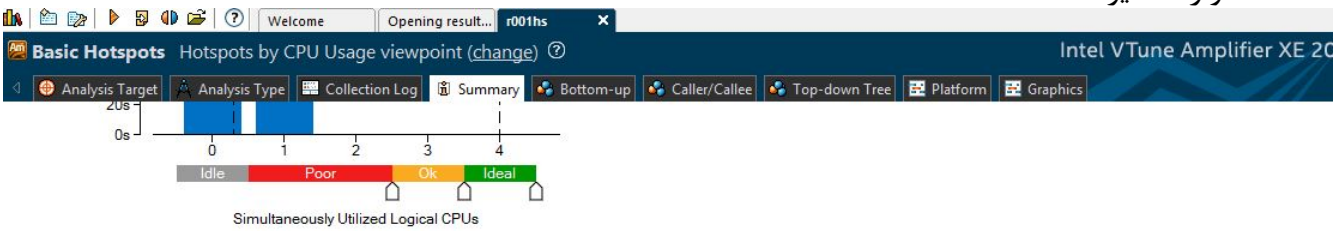


CPU Usage Histogram

This histogram displays a percentage of the wall time the specific number of CPUs were running simultaneously. Spin and Overhead time adds to the Idle CPU usage value.



خلاصه موارد آنالیز:



Collection and Platform Info

This section provides information about this collection, including result set size and collection platform data.

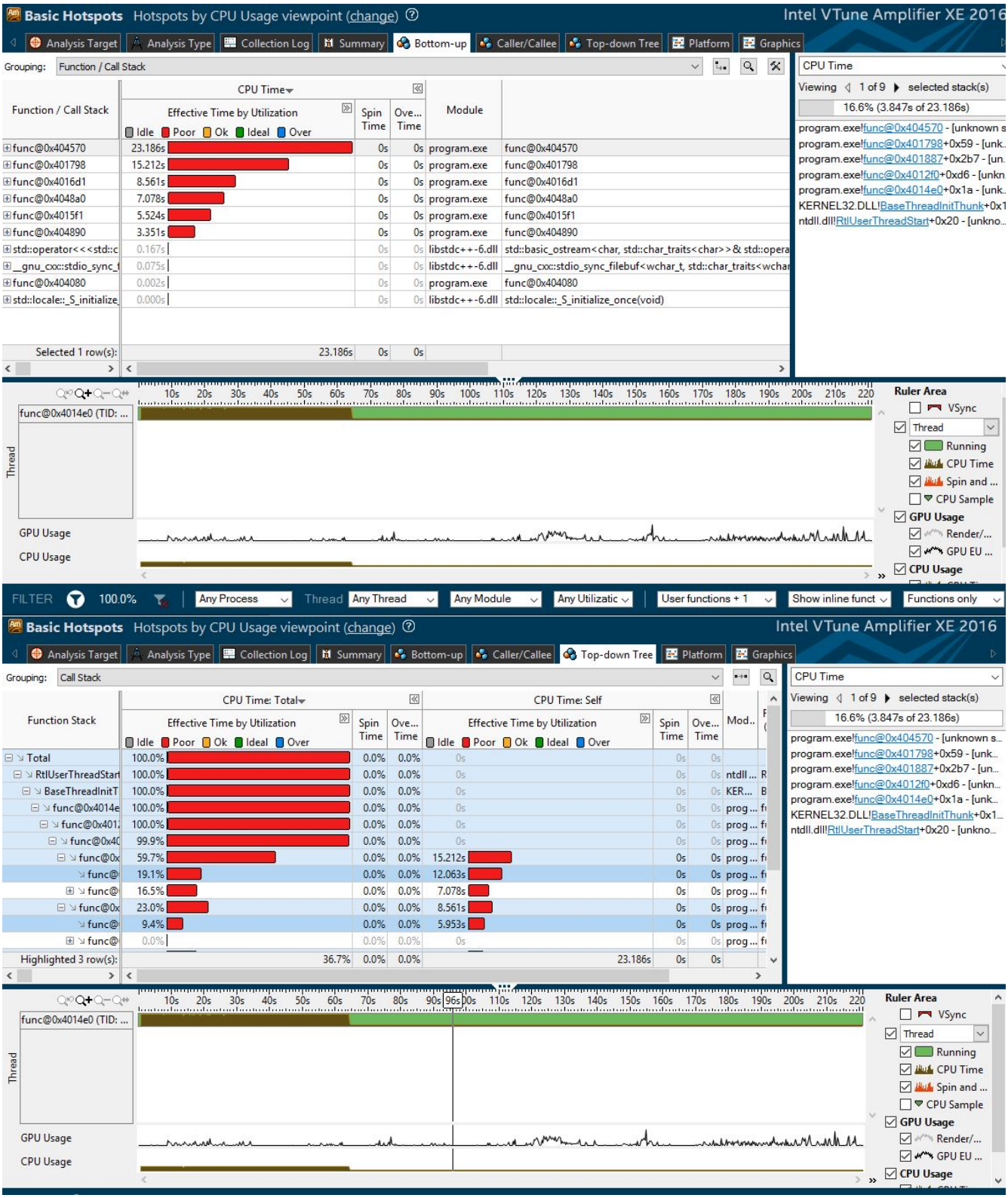
Application Command Line:	C:\Users\SAA\Desktop\New folder (2)\program.exe
Operating System:	Microsoft Windows 10
Computer Name:	DESKTOP-JD3MVGS
Result Size:	307 MB
Collection start time:	20:20:40 26/06/2024 UTC
Collection stop time:	20:24:21 26/06/2024 UTC

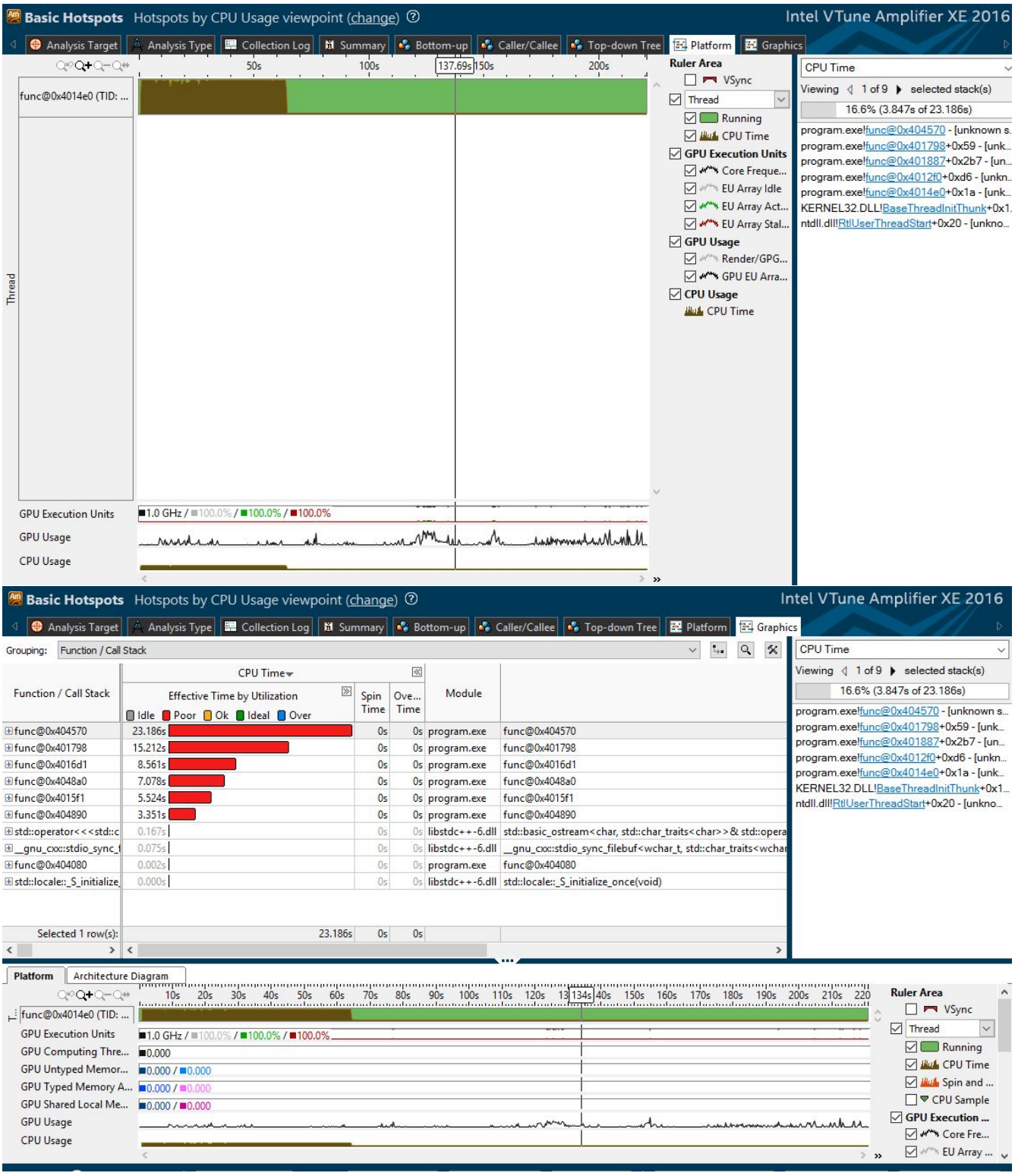
CPU

Name:	4th generation Intel(R) Core(TM) Processor family
Frequency:	2.3 GHz
Logical CPU Count:	4

GPU

Name:	NVIDIA GeForce GT 720M
Vendor:	NVIDIA
Driver:	9.18.13.2683 (30/05/1392)
Stepping:	161
EU Count:	20
Max EU Thread Count:	7
Max Core Frequency:	1.0 GHz


Basic Hotspots Hotspots by CPU Usage viewpoint (change)
Intel VTune Amplifier XE 2016
Analysis Target
Analysis Type
Collection Log
Summary
Bottom-up
Caller/Callee
Top-down Tree
Platform
Graphics
Grouping: Function / Call Stack
Function / Call Stack
CPU Time
Effective Time by Utilization
Idle Poor Ok Ideal Over
Spin Time
Ove... Time
Module
func@0x404570 23.186s 0s 0s program.exe func@0x404570
func@0x401798 15.212s 0s 0s program.exe func@0x401798
func@0x4016d1 8.561s 0s 0s program.exe func@0x4016d1
func@0x4048a0 7.078s 0s 0s program.exe func@0x4048a0
func@0x4015f1 5.524s 0s 0s program.exe func@0x4015f1
func@0x404890 3.351s 0s 0s program.exe func@0x404890
std::operator<<<std::c 0.167s 0s 0s libstdc++-6.dll std::basic_ostream<char, std::char_traits<char>>& std::opera
__gnu_cxx::stdio_sync_t 0.075s 0s 0s libstdc++-6.dll __gnu_cxx::stdio_sync_filebuf<wchar_t, std::char_traits<wchar
func@0x404080 0.002s 0s 0s program.exe func@0x404080
std::locale::_S_initialize 0.000s 0s 0s libstdc++-6.dll std::locale::_S_initialize_once(void)
Selected 1 row(s): 23.186s 0s 0s
CPU Time
Viewing 1 of 9 selected stack(s)
16.6% (3.847s of 23.186s)
program.exe!func@0x404570 - [unknown s
program.exe!func@0x401798+0x59 - [unk..
program.exe!func@0x401887+0x2b7 - [un..
program.exe!func@0x4012f0+0xd6 - [unkn
program.exe!func@0x4014e0+0x1a - [unk..
KERNEL32.DLL!BaseThreadInitThunk+0x1
ntdll.dll!RtlUserThreadStart+0x20 - [unkno..
10s 20s 30s 40s 50s 60s 70s 80s 90s 100s 110s 120s 130s 140s 150s 160s 170s 180s 190s 200s 210s 220
func@0x4014e0 (TID: ...
Thread
GPU Usage
CPU Usage
Ruler Area
VSync
Thread
Running
CPU Time
Spin and ...
CPU Sample
GPU Usage
Render/...
GPU EU ...
CPU Usage
FILTER 100.0%
Any Process
Thread Any Thread
Any Module
Any Utilizatic
User functions + 1
Show inline funct
Functions only
Basic Hotspots Hotspots by CPU Usage viewpoint (change)
Intel VTune Amplifier XE 2016
Analysis Target
Analysis Type
Collection Log
Summary
Bottom-up
Caller/Callee
Top-down Tree
Platform
Graphics
Grouping: Call Stack
Function Stack
CPU Time: Total
CPU Time: Self
Effective Time by Utilization
Idle Poor Ok Ideal Over
Spin Time
Ove... Time
Mod..
Total 100.0% 0.0% 0.0%
RtlUserThreadStart 100.0% 0.0% 0.0% ntdll ...
BaseThreadInitT 100.0% 0.0% 0.0% KER...
func@0x4014e 100.0% 0.0% 0.0% prog ...
func@0x401 100.0% 0.0% 0.0% prog ...
func@0x40 99.9% 0.0% 0.0% prog ...
func@0x 59.7% 0.0% 0.0% 15.212s 0s 0s prog ...
func@ 19.1% 0.0% 0.0% 12.063s 0s 0s prog ...
func@ 16.5% 0.0% 0.0% 7.078s 0s 0s prog ...
func@0x 23.0% 0.0% 0.0% 8.561s 0s 0s prog ...
func@ 9.4% 0.0% 0.0% 5.953s 0s 0s prog ...
func@ 0.0% 0.0% 0.0% 0s 0s 0s prog ...
Highlighted 3 row(s): 36.7% 0.0% 0.0% 23.186s 0s 0s
CPU Time
Viewing 1 of 9 selected stack(s)
16.6% (3.847s of 23.186s)
program.exe!func@0x404570 - [unknown s...
program.exe!func@0x401798+0x59 - [unk...
program.exe!func@0x401887+0x2b7 - [un...
program.exe!func@0x4012f0+0xd6 - [unkn...
program.exe!func@0x4014e0+0x1a - [unk...
KERNEL32.DLL!BaseThreadInitThunk+0x1...
ntdll.dll!RtlUserThreadStart+0x20 - [unkno...
10s 20s 30s 40s 50s 60s 70s 80s 90s 96s 00s 110s 120s 130s 140s 150s 160s 170s 180s 190s 200s 210s 220
func@0x4014e0 (TID: ...
Thread
GPU Usage
CPU Usage
Ruler Area
VSync
Thread
Running
CPU Time
Spin and ...
CPU Sample
GPU Usage
Render/...
GPU EU ...
CPU Usage


Basic Hotspots Hotspots by CPU Usage viewpoint (change)
Intel VTune Amplifier XE 2016
Analysis Target
Analysis Type
Collection Log
Summary
Bottom-up
Caller/Callee
Top-down Tree
Platform
Graphics
Ruler Area
VSync
Thread
Running
CPU Time
GPU Execution Units
Core Freque...
EU Array Idle
EU Array Act...
EU Array Stal...
GPU Usage
Render/GPG...
GPU EU Arra...
CPU Usage
CPU Time
Viewing 1 of 9 selected stack(s)
16.6% (3.847s of 23.186s)
program.exe!func@0x404570 - [unknown s...
program.exe!func@0x401798+0x59 - [unk...
program.exe!func@0x401887+0x2b7 - [un...
program.exe!func@0x4012f0+0xd6 - [unkn...
program.exe!func@0x4014e0+0x1a - [unk...
KERNEL32.DLL!BaseThreadInitThunk+0x1...
ntdll.dll!RtlUserThreadStart+0x20 - [unkno...
Grouping: Function / Call Stack
Function / Call Stack
CPU Time
Effective Time by Utilization
Idle Poor Ok Ideal Over
Spin Time
Ove... Time
Module
func@0x404570 23.186s 0s 0s program.exe
func@0x401798 15.212s 0s 0s program.exe
func@0x4016d1 8.561s 0s 0s program.exe
func@0x4048a0 7.078s 0s 0s program.exe
func@0x4015f1 5.524s 0s 0s program.exe
func@0x404890 3.351s 0s 0s program.exe
std::operator<<<std::c 0.167s 0s 0s libstdc++-6.dll
__gnu_cxx::stdio_sync_t 0.075s 0s 0s libstdc++-6.dll
func@0x404080 0.002s 0s 0s program.exe
std::locale::_S_initialize 0.000s 0s 0s libstdc++-6.dll
Selected 1 row(s): 23.186s 0s 0s
Platform
Architecture Diagram
func@0x4014e0 (TID: ...
GPU Execution Units
GPU Computing Thre...
GPU Untyped Memor...
GPU Typed Memory A...
GPU Shared Local Me...
GPU Usage
CPU Usage
Spin and ...
CPU Sample
GPU Execution ...
Core Fre...
EU Array ...